Bancos de Dados Não-Relacionais

Aula 07: CRUD Avançado, Atualizações Atômicas e Indexação de Alta Performance no MongoDB

A presentation by
Prof. Me. Reginaldo Fernandes @ Instituto Federal do Ceará – Campus Tauá Tecnologia em Análise e Desenvolvimento de Sistemas

03 de Novembro de 2026

INSTITUTO FEDERAL
Ceará
Campus Tauá

Agenda do Encontro

Contextualização Prática: Gargalos de Escala e a Teoria da *B*-Tree

CRUD Avançado: Filtros Complexos e Atualizações Atômicas

Teoria de Índices: Índices Simples, Compostos e Otimizações de Memória

A Regra de Ouro ESR, Índices Multikey e Busca Textual

Diagnóstico com `explain()`, *Covered Queries* e Benchmark

Contextualização Prática: Gargalos de Escala e a Teoria da *B*-Tree

O Problema de Escala no E-commerce ShopGrids

O Cenário sob Concorrência Real

- Na Aula 06, estruturamos os esquemas BSON do **ShopGrids** com documentos ricos.
- Em ambiente de desenvolvimento com 50 itens, qualquer consulta parece instantânea ($\leq$ 1 ms).
- **O Ponto de Ruptura:** Ao carregar 50.000 produtos e 10.000 pedidos sob tráfego concorrente, consultas sem índice geram colapso de I/O e CPU em 100%!

A Anatomia do `COLLSCAN` (*Collection Scan*)

- Sem índice, o motor WiredTiger realiza uma varredura completa da coleção: **`COLLSCAN`**.
- **Complexidade Temporal:** $\mathcal{O}(N)$, onde N é o **Tamanho da Coleção**.
- Cada documento BSON precisa ser carregado do disco, descompactado na memória RAM e inspecionado campo por campo.

A Estrutura de Árvore Balanceada (*B*-Tree) no WiredTiger

Como o MongoDB Organiza os Índices

- Os índices no MongoDB são estruturados como **Árvores Balanceadas (*B*-Tree)**.
- **Nó Raiz (Root):** Ponto de entrada com chaves ordenadas e ponteiros.
- **Nós Internos:** Direcionam a busca em profundidade através de bifurcações.
- **Nós Folha (Leaves):** Contêm os valores indexados e o ponteiro físico direto para o documento (*RecordId*).

Fator de Ramificação e Complexidade

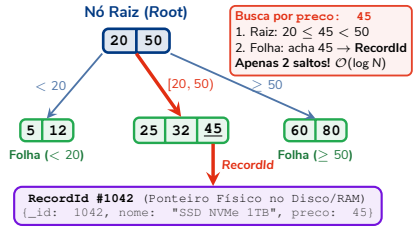
- **Fator de Ramificação (*b*):** Número de filhos por nó (típico de 50 a 200 entradas em páginas de 4 KB).
- **Complexidade Logarítmica:** $\mathcal{O}(\log_b N)$, onde $h \approx \lceil \log_b(N) \rceil$ denota a **Altura da Árvore**.
- Em vez de ler *N* documentos, a busca descende por apenas *h* nós balanceados na memória RAM!

Fundamentos da Árvore Balanceada (*B*-Tree)

O Que É uma Árvore *B*-Tree?

- **Conceito:** Estrutura de busca auto-balanceada multidirecional (m-vias), projetada para operações eficientes em blocos de disco/SSD e memória RAM.
- **Nós com Múltiplas Chaves:** Ao contrário de árvores binárias (1 chave, 2 filhos), nós de *B*-Tree contêm dezenas a centenas de chaves ordenadas ($k_1 < k_2 < \cdots < k_n$) e direcionam múltiplos filhos ($n + 1$).
- **Balanceamento Perfeito:** Todas as folhas residem rigorosamente na **mesma profundidade** (h). Divisões de nós ocorrem para cima, garantindo busca sempre em $\mathcal{O}(\log N)$.
- **Roteamento por Faixas:** As chaves funcionam como balizas ordenadas, saltando diretamente ao bloco alvo sem varredura sequencial.

Anatomia Visual e Mecanismo de Busca



Exemplo Matemático Pareado: $\mathcal{O}(N)$ vs $\mathcal{O}(\log_b N)$

1. Cálculo Analítico Passo a Passo

Considere a coleção com $N = 100.000$ produtos e fator $b = 100$:

- **Sem Índice (`COLLSCAN`):**

$$\text{Acessos}_{\text{scan}} = N = 100.000 \text{ documentos lidos}$$

- **Com Índice *B*-Tree (`IXSCAN`):**

$$h \approx \lceil \log_{100}(100.000) \rceil = \lceil 2.5 \rceil = 3$$

- **Acessos necessários:** Apenas **3 nós** inspecionados!
- **Redução de I/O:**

$$\left(1 - \frac{3}{100.000}\right) \times 100\% = \mathbf{99.997\%}$$

2. Interpretação em Hardware

- A busca sequencial exige ler megabytes de dados brutos e satura a banda do barramento PCIe/SATA.
- A *B*-Tree realiza saltos diretos em blocos ordenados.
- Como a raiz e os nós intermediários residem no cache WiredTiger na RAM, o acesso a um registro é quase instantâneo ($\leq$ 0.5 ms).

Demonstração Computacional Pareada em Python

Implementação em Python medindo a diferença prática entre busca linear e busca em árvore:

```python
import time, bisect

N = 100_000
dados_lineares = list(range(N))
dados_ordenados = sorted(dados_lineares) # Estrutura do Indice B-Tree
alvo = 99_999 # Pior caso (elemento final)

# 1. Busca Linear O(N) -> Simulacao de COLLSCAN
t0 = time.perf_counter()
for item in dados_lineares:
    if item == alvo: break
tempo_scan = (time.perf_counter() - t0) * 1000

# 2. Busca em Arvore O(log N) -> Simulacao de IXSCAN
t0 = time.perf_counter()
pos = bisect.bisect_left(dados_ordenados, alvo)
tempo_indice = (time.perf_counter() - t0) * 1000

print(f"COLLSCAN (Linear): {tempo_scan:.4f} ms")
print(f"IXSCAN (Indice)  : {tempo_indice:.4f} ms")
# Resultado: IXSCAN eh mais de 800x mais rapido com N = 100.000!
```

CRUD Avançado: Filtros Complexos e Atualizações Atômicas

O Operador `$elemMatch` e o Problema dos Falsos Positivos

A Armadilha do Filtro Ingênuo em Arrays

Considere o pedido com dois itens:

- Item A: `preco = 250`, `qtd = 1`
- Item B: `preco = 20`, `qtd = 8`

Se consultarmos:

```
db.pedidos.find({
  "itens.preco": { $gt: 100 },
  "itens.qtd": { $gte: 5 }
})
```

Resultado: O documento é retornado! O Item A atendeu ao preço e o Item B à quantidade. O filtro avaliou itens **diferentes**!

A Solução Estrita com `$elemMatch`

O operador `$elemMatch` força o motor a exigir que **todas as condições sejam atendidas no mesmo elemento**:

```
db.pedidos.find({
  itens: {
    $elemMatch: {
      preco: { $gt: 100 },
      qtd: { $gte: 5 }
    }
  }
})
```

Resultado: Nenhum documento é retornado, eliminando com precisão o falso positivo!

Operador `$expr`: Comparação de Campos Internos

Quando Usar o `$expr`?

- Filtros comuns comparam um campo com um valor estático (`preco > 100`).
- Para comparar **dois campos do mesmo documento**, usamos `$expr`.
- O prefixo `$` indica que o valor daquele campo deve ser resolvido em tempo de execução.

Exemplos Práticos no E-Commerce

```
// 1. Produtos com estoque atual abaixo do
      estoque minimo:
db.produtos.find({
  $expr: {
    $lt: ["$estoque.atual",
          "$estoque.minimo"]
  }
})

// 2. Pedidos em que total pago foi menor
      que o subtotal:
db.pedidos.find({
  $expr: {
    $lt: ["$pagamento.total_pago",
          "$pagamento.subtotal"]
  }
})
```

Atualizações Atômicas Concorrentes: `$inc`, `$min` e `$max`

Por que Evitar `$set` para Contadores?

- Ler o estoque para a memória da aplicação, subtrair 1 e salvar com `$set` causa **Condições de Corrida (*Race Conditions*)**.
- Se duas requisições concorrentes lerem estoque 10, ambas gravarão 9, perdendo uma venda!
- O operador **`$inc`** opera diretamente no bloco de memória do WiredTiger com garantia de atomicidade.

Sintaxe de Operadores Atômicos

```
// Decrementar estoque de forma atomica:
db.produtos.updateOne(
  { sku: "SKU-INF-000101" },
  { $inc: { "estoque.atual": -2 } }
)

// Atualizar apenas se for o menor preco
      historico:
db.produtos.updateOne(
  { sku: "SKU-INF-000101" },
  { $min: { "historico.menor_preco": 299.90
        } }
)
```

Manipulação de Arrays: `$push`, `$slice` e `arrayFilters`

Manutenção do *Subset Pattern*

Ao inserir avaliações ou logs, usamos `$push` com o modificador `$slice`: `-5` para manter o array limitado aos últimos 5 itens:

```
db.produtos.updateOne(
  { sku: "SKU-INF-000101" },
  {
    $push: {
      ultimas_avaliacoes: {
        $each: [{ nota: 5, autor: "Maria" }],
        $slice: -5 // Mantem apenas os 5
                   mais recentes
      }
    }
  }
)
```

Modificação Seletiva com `arrayFilters`

Atualiza um subdocumento específico de um array usando o identificador `$[alvo]`:

```
db.pedidos.updateOne(
  { codigo: "PED-2026-001" },
  {
    $set: {
      "itens.$[alvo].status_entrega":
            "SEPARADO",
      "itens.$[alvo].data": new Date()
    }
  },
  {
    arrayFilters: [ { "alvo.sku": "ITEM-A" }
          ]
  }
)
```

Teoria de Índices: Índices Simples, Compostos e Otimizações de Memória

Gestão e Anatomia de Índices no MongoDB

Comandos Essenciais no Terminal

- O MongoDB cria automaticamente o índice único no campo `_id`.
- Todo índice adicional consome espaço em disco e memória RAM no cache do WiredTiger.

```
// Listar indices existentes:
db.produtos.getIndexes()

// Remover um indice pelo nome:
db.produtos.dropIndex("idx_sku_1")
```

Índice Simples (*Single Field Index*)

```
db.produtos.createIndex(
  { sku: 1 },
  { unique: true, name: "idx_sku_unico" }
)
```

- **Direção (1 vs -1):** Em índices simples, a direção é irrelevante para a performance, pois o motor pode percorrer a *B*-Tree bidirecionalmente com o mesmo custo.
- **`unique: true`:** Garante unicidade e integridade no banco.

Índices Compostos e a Regra dos Prefixos

O que é um Índice Composto?

- Índice construído sobre dois ou mais campos da coleção.
- A ordenação é hierárquica: os nós são ordenados pelo primeiro campo; em caso de empate, pelo segundo, e assim por diante.

```
db.produtos.createIndex({
  categoria: 1,
  preco: -1
})
```

Correspondência de Prefixos (*Index Prefixes*)

Considere o índice composto em `{ A: 1, B: 1, C: 1 }`:

- **Prefixos Válidos (Acelerados):**
  - Consulta sobre `{ A }`
  - Consulta sobre `{ A, B }`
  - Consulta sobre `{ A, B, C }`
- **Consultas NÃO Aceleradas:**
  - Consulta sobre `{ B }` ou `{ C }`
  - Consulta sobre `{ B, C }` sem o campo `A`

Otimizações de Memória RAM: Índices Parciais, Esparsos e TTL

Índices Parciais (*Partial Indexes*)

Indexam apenas documentos que atendem a um filtro, economizando dezenas de megabytes no cache da RAM:

```
// Indexa apenas pedidos PENDENTES:
db.pedidos.createIndex(
  { cliente_id: 1, data_criacao: -1 },
  { partialFilterExpression: { status:
        "PENDENTE" } }
)
```

- Pedidos já entregues ou cancelados não ocupam espaço na árvore balanceada!

Índices TTL (*Time-To-Live*)

Excluem automaticamente documentos expirados do banco sem necessidade de rotinas de *cron job*:

```
// Exclui o carrinho apos 3600 segundos:
db.carrinhos.createIndex(
  { data_expiracao: 1 },
  { expireAfterSeconds: 0 }
)
```

- O motor WiredTiger roda uma thread em segundo plano a cada 60s para limpeza automática.

A Regra de Ouro ESR, Índices Multikey e Busca Textual

A Regra de Ouro ESR (*Equality, Sort, Range*)

A Ordem Mandatória dos Campos

Ao desenhar índices compostos que misturam filtros, ordenação e faixas, siga sempre a ordem:

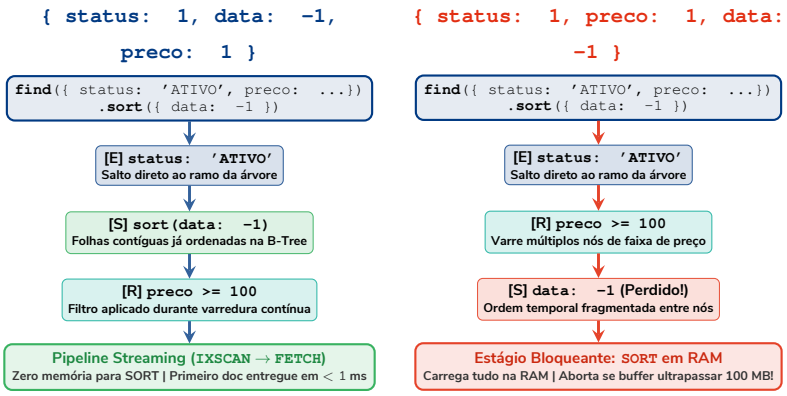
1. **[E] Equality (Igualdade):** Campos filtrados por igualdade exata (`status: 'ATIVO'`).
2. **[S] Sort (Ordenação):** Campos da cláusula `sort` (`data_cadastro: -1`).
3. **[R] Range (Faixa):** Campos com operadores de desigualdade (`preco: { $gte: 100 }`).

```
// Indice Composto Ideal (E-S-R):
db.produtos.createIndex({
  status: 1,          // [E]
  data_cadastro: -1,  // [S]
  preco: 1            // [R]
})
```

O Perigo Fatal de Quebrar a Regra ESR

- Se colocarmos a Faixa antes do Sort (**E-R-S**): o motor varre os nós da faixa de preço, mas perde a ordenação temporal contígua da *B*-Tree!
- O banco é obrigado a executar uma etapa custosa de **`SORT` em memória RAM**.
- **Erro Fatal:** Se o volume de dados para ordenação em memória ultrapassar **100 MB**, o MongoDB aborta a consulta com erro de execução: *Sort exceeded memory limit of 104857600 bytes*!

Diagrama Arquitetural: Fluxo de Execução da Regra ESR



Índices Multikey (Indexando Listas e Arrays)

Como Funciona o Índice Multikey?

- Quando um campo armazena um array (ex: `tags: ['gamer', 'rgb', 'usb-c']`), o MongoDB cria um índice **Multikey**.
- O WiredTiger gera uma entrada individual na árvore para cada elemento do array.

```
db.produtos.createIndex({ tags: 1 })

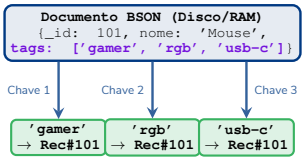
// Consulta acelerada pelo Multikey:
db.produtos.find({ tags: "sem fio" })
```

[Atenção] A Restrição de Arrays Paralelos

- Em um índice composto, **no máximo um dos campos indexados pode ser array**!
- Tentar criar índice em `{ tags: 1, cores: 1 }` onde ambos sejam listas causa o erro fatal: *cannot index parallel arrays*!
- **Razão Matemática:** Evitar a explosão combinatória cartesiana ($M \times N$ entradas no índice por documento).

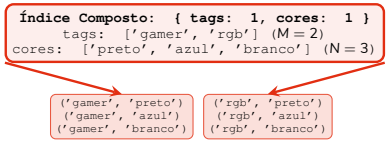
Diagrama Arquitetural: Mecanismo Interno do Índice Multikey

Indexação de Arrays (Desmembramento 1:N)



Propriedades do Multikey:
- Cada elemento gera uma chave na B-Tree.
- Busca `find({ tags: 'rgb' })` em $\mathcal{O}(\log N)$.
- Deduplicação automática na entrega.

Por Que Arrays Paralelos São Proibidos?



Explosão Combinatória: $M \times N = 6$ **nós/doc!**
Com 10 tags e 10 cores $\Longrightarrow$ 100 nós por doc!
Bloqueio: *cannot index parallel arrays*

Índices Textuais Ponderados (*Full-Text Search*)

Criação com Pesos (*Weights*)

Atribui prioridade matemática ao título em relação à descrição e ativa o analisador de língua portuguesa:

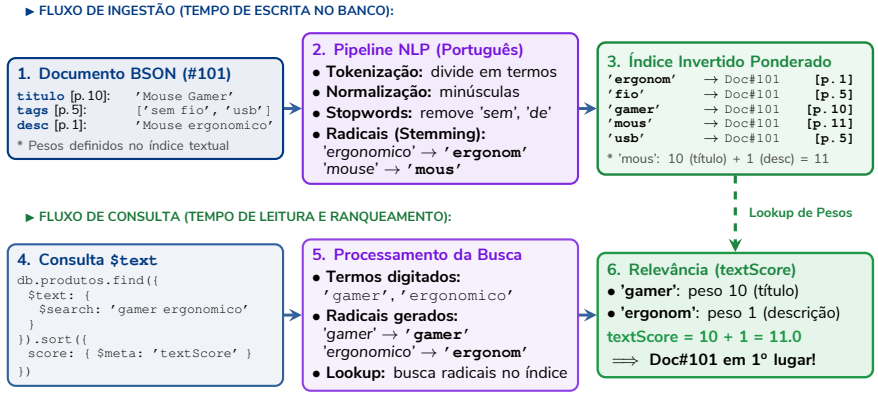
```
db.produtos.createIndex(
  {
    titulo: "text",
    tags: "text",
    descricao: "text"
  },
  {
    weights: { titulo: 10, tags: 5,
           descricao: 1 },
    default_language: "portuguese",
    name: "idx_busca_textual"
  }
)
```

Busca e Ranqueamento por Escore

Executa a busca com radicalização (*stemming*) e ordena os resultados por relevância métrica:

```
db.produtos.find(
  { $text: { $search: "ergonomico sem fio" }
        },
  {
    titulo: 1,
    preco: 1,
    score: { $meta: "textScore" },
    _id: 0
  }
).sort({ score: { $meta: "textScore" } })
```

Diagrama Arquitetural: Pipeline do Índice Invertido Textual



Diagnóstico com `explain()`, *Covered Queries* e Benchmark

Auditoria de Execução: Os Modos do `explain()`

Níveis de Auditoria do Otimizador

- **`queryPlanner` (Padrão):** Retorna apenas o plano eleito pelo otimizador sem executar a consulta.
- **`executionStats` (Recomendado):** Executa a consulta e retorna métricas reais de tempo, nós examinados e documentos lidos.
- **`allPlansExecution`:** Executa e documenta o desempenho de todos os planos candidatos testados.

Sintaxe no MongoSH / PyMongo

```
// Executando auditoria completa:
db.produtos.find(
  { sku: "SKU-INF-000101" }
).explain("executionStats")
```

- Permite aos engenheiros de dados inspecionar se a consulta está usando índice (`IXSCAN`) ou colapsando em `COLLSCAN`.

Decodificando as Quatro Métricas Vitais do `executionStats`

Métrica no JSON	Conceito Técnico	Alvo de Otimização
`stage`	Estágio da árvore de execução.	`IXSCAN` (Sem `COLLSCAN` ou `SORT` em RAM).
`totalDocsExamined`	Documentos físicos lidos do disco/cache.	Deve aproximar-se de `nReturned` (ou 0).
`totalKeysExamined`	Nós da *B*-Tree inspecionados.	Próximo a `nReturned` (afunilamento eficiente).
`nReturned`	Documentos realmente devolvidos.	Volume útil solicitado pela aplicação.
`executionTimeMillis`	Tempo de processamento em ms.	$\leq$ 1 ms para operações pontuais.

Razão de Eficiência:

$$\frac{\text{totalDocsExamined}}{\text{nReturned}} \quad (\text{Ideal} = 1.0; \quad \text{COLLSCAN sob } 50.000 \text{ itens} = 5.000).$$

Covered Queries (Consultas Cobertas): O Santo Graal da Performance

O que é uma Consulta Coberta?

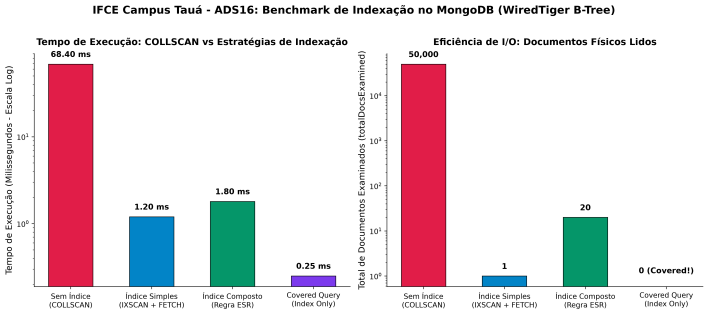
- Todos os campos da consulta (filtro e projeção) estão presentes no próprio índice *B*-Tree.
- O MongoDB **NUNCA lê o documento BSON do disco**!
- O estágio **`FETCH`** desaparece, operando 100% em memória RAM com **`totalDocsExamined: 0`**!

Código da Consulta Coberta

```
// 1. Criar indice cobrindo categoria, preco
      e sku:
db.produtos.createIndex({
  categoria: 1,
  preco: 1,
  sku: 1
})

// 2. Consulta 100% coberta:
// ATENCAO: _id: 0 eh OBRIGATORIO para
      evitar o FETCH!
db.produtos.find(
  { categoria: "Informática", preco: { $lte:
        1500 } },
  { categoria: 1, preco: 1, sku: 1, _id: 0 }
).explain("executionStats")
```

Resultados do Benchmark no Laboratório (50.000 Documentos)



Conclusão: Adoção de índices compostos (ESR) e consultas cobertas reduz a latência em mais de 99.7% e zera leituras físicas de documentos.

Dúvidas & Prática em Laboratório

IFCE Campus Tauá – ADS16 (Modelo Orientado a Documentos)

CRUD Avançado, Índices *B*-Tree, Regra ESR e Consultas Cobertas